

Oak GTM Automation - Live Event Lead Engine

Ten n8n workflows that take a booth badge scan and turn it into a routed, enriched, CRM-synced lead with a Slack card a rep can act on, in about two seconds of scanner-visible latency.

This repository holds the exported workflow JSON. **The export is not one-click importable, and this document says exactly why and what standing it up requires.** I would rather hand over honest prerequisites than a file that imports 60% of the way and then fails quietly.

The workflows

File	Role	Trigger
00 Setup & Seed	Creates the six Data Tables, seeds the policy, competitors and test payloads, provisions the HubSpot properties	Manual, run once
01 Intake & Decision	The orchestrator. Booth webhook lands here; classification, scoring and routing are decided here	Webhook
02 HubSpot Sync	The only place the automated pipeline writes to the CRM	Called by 01
02A CRM Context	The read half of that pair. Every call is a GET or a search	Called by 01
03 Slack Alerts	Renders and delivers the card, handles threading for returning visitors	Called by 01
04 Company & Person Enrichment	Homepage read plus gated public search; headcount and identity evidence	Called by 01
05 AI Lead Reasoning	Bounded LLM step: persona read on ambiguous titles, opening question, listen-fors	Called by 01
06 Slack Actions	Twelve rep buttons and four modals. Writes to HubSpot only on a human tap	Slack interactivity webhook
07 Task Test Harness	Replays payloads through the live webhook and reports PASS/FAIL per scan	Manual
99 Failure Handler	Shared error workflow. Named by the eight other active workflows	n8n error trigger

Only 01 and 06 are reachable from outside. 00 and 07 are manual. 99 has no caller at all - n8n invokes it when a workflow that names it fails.

Standing it up in a fresh n8n

Four steps, in this order. Steps 1 and 2 are the ones most likely to waste your time.

1. Create the Data Tables

Run `00 Setup & Seed` first. It creates six tables by name with `createIfNotExists` , then seeds the active policy row, 17 competitor identities and 20 test payloads.

Then add one column by hand: `oak_deliveries.slack_json` . Workflow 00 provisions 17 columns and does not include it, but `01` writes it and `07` reads it back to assert Slack delivery. Because the create nodes use `createIfNotExists` , re-running `00` will never add a missing column - so this step cannot be skipped by running the seeder again.

Known limitation worth stating plainly: `00` 's idempotence is name-based, not shape-aware. It creates what is missing and never inspects what already exists. A table whose columns diverged, or a HubSpot property whose type diverged, is neither detected nor repaired. See the note on `oak_competitor_status` below for a real case.

2. Re-pick the five sub-workflows in `01`

`01` calls its five sub-workflows by hardcoded ID:

Node in <code>01</code>	Calls
<code>Read CRM Context</code>	<code>02A</code>
<code>Enrich Company</code>	<code>04</code>
<code>Reason with AI</code>	<code>05</code>
<code>Sync to HubSpot</code>	<code>02</code>
<code>Notify Slack</code>	<code>03</code>

n8n does not remap these on import, and there is no name mode for the Execute Sub-workflow node. Worse, all five carry a `cachedResultName` , so after import the canvas displays the *correct* sub-workflow name next to a dead ID. It looks right and is not. Open each of the five and re-pick the workflow from the list.

3. Re-point the Data Table nodes

Twenty nodes reference their table by this instance's table ID rather than by name:

Workflow	Pinned nodes
<code>01</code>	10
<code>03</code>	2
<code>04</code>	3
<code>05</code>	1
<code>07</code>	4

Either re-pick the table in each, or switch the resource locator to **name** mode, which `06` already uses in production for all four of its Data Table nodes. Name mode works for every operation this build uses - `get` ,

`upsert` , `update` , `deleteRows` and `create` - and a misspelled name fails loudly (`NodeOperationError: Data table with name "..."` not found) rather than silently returning nothing.

4. Credentials, variables and settings

Three credentials, note that two are Slack of different types:

Credential	Type	Used by
HubSpot	<code>hubspot0Auth2Api</code>	<code>00</code> , <code>02</code> , <code>02A</code> , <code>06</code>
Slack	<code>slack0Auth2Api</code>	<code>00</code> , <code>99</code>
Slack (app)	<code>slackApi</code>	<code>03</code> , <code>06</code>

Gemini and Brave have no credential object. They run on n8n Cloud gateway credits. On a fresh Cloud instance with credits they work untouched. Self-hosted needs real API keys for both, plus the community package `@brave/n8n-nodes-brave-search` installed - without it `04` 's three search nodes will not load at all.

Three n8n variables:

Variable	Read by	Behaviour when unset
<code>OAK_WEBHOOK_SECRET</code>	<code>07</code> to send, <code>01</code> to verify	Fails open. The booth webhook accepts anything and reports <code>auth_mode: open_no_secret_configured</code>
<code>OAK_SLACK_SIGNING_SECRET</code>	<code>06</code>	Fails closed. Every Slack button stops working
<code>OAK_SLACK_ACTIONS_ENABLED</code>	<code>03</code> , <code>06</code>	Buttons are not rendered

The opposite defaults are deliberate: a booth scan must never be lost because a secret was not configured, and a CRM write must never happen on an unverified request.

Also: re-pick `99` as the error workflow on the eight workflows that name it (`01` , `02` , `02A` , `03` , `04` , `05` , `06` , `07`), and edit `WEBHOOK_URL` at the top of `07` 's `Build Test Plan` to point at your own `01` webhook.

Slack channels are provisioned by workflow `00` , not by hand. `Plan Slack Channels` reads the names out of the `notifications` block of the active `oak_policies` row and `Create Slack Channel` creates all six - `#booth-hot` , `#booth-matched` , `#booth-review` , `#competitive-intel` (private), `#booth-out-of-scope` and `#oak-revops` - so changing a channel in the policy changes what setup creates, with no workflow edit. The Slack credential needs `channels:manage` and `groups:write` . A rerun is clean: a channel that already exists comes back `name_taken` and is counted as already provisioned.

The one manual step. Setup creates channels; it does not join them. For any of the six that already existed in your workspace, the Slack app is not a member and `00` will not tell you, because it counts `name_taken` as success. Invite the app to each pre-existing channel. `#competitive-intel` is private, so a member has to invite the app

- it cannot join itself. The symptom of missing this is a `not_in_channel` error

recorded in `oak_deliveries.slack_json` and no card in the channel. Workflow `07` fails the run on it, but a rep at the booth just sees nothing.

One trap worth knowing about

The live HubSpot portal's `oak_competitor_status` is an **enumeration** accepting only `none`, `possible`, `confirmed`. Workflow `00` provisions it as free text.

Both are true because `00` only creates properties the portal is missing, so a field that already exists with a different type is never reconciled. The consequence for a rebuild: **on a fresh portal the field is created as free text, and the enum constraint the code respects will exist nowhere in your instance**. The failure it guards against will not reproduce, which makes it harder to find later, not easier.

That constraint is load-bearing. All 18 `oak_*` properties go out in a single PATCH, so one rejected value discards all 18 - tier, persona, score and reasons included. That is not hypothetical; it happened, and HubSpot answered:

```
{
  "status": "error",
  "category": "VALIDATION_ERROR",
  "message": "Property values were not valid: [{\n\"isValid\": false,\n  \"message\": \"confirmed: Veza was not one of the allowed options: [none, possible, confirmed]\n  \"error\": \"INVALID_OPTION\", \"name\": \"oak_competitor_status\"}]"}
}
```

It is only visible at all because `crm_context.oak_properties_written` is asserted from the response body rather than assumed from the node having run. A competitor's actual name belongs in the booth note, not in an enum.

Verifying it works

`07 Task Test Harness` runs the four task leads end to end against the live webhook and reports PASS/FAIL per scan. Pick which scans run by toggling `enabled` in the `oak_test_payloads` Data Table - 20 rows across nine suites, with the four task leads enabled by default. The four payloads are also embedded in `07` as a fallback for a run against an empty table.

A PASS asserts, in order: the webhook accepted the scan, an `oak_deliveries` row exists, its status reached `complete`, Slack delivery succeeded where the stored decision named a channel, and only then the expected classification, persona and channel. A row with `expect.accepted: false` inverts the first two - the webhook must reject, and no row may exist.

Design notes

Policy as data. Channels, thresholds, personas, competitor terms, the ICP and the confidence floor live in a row in `oak_policies`, read at runtime. Editing that row changes behaviour with no redeploy. Re-running `00` would overwrite a hand-edit.

Deterministic routing, bounded AI. The tier, the score and the channel are computed in code against the policy. The model's one job is reading a job title the rule matcher could not classify, above a confidence floor. A resolved persona changes the classification, and therefore the channel - so a model can change which policy row is read, but never write the row, invent a channel, or add a mention.

Everything soft-fails except the webhook. Enrichment, AI, HubSpot and Slack all carry `onError: continueRegularOutput`, because a booth scanner cannot wait and a partial lead beats a lost one. 99 catches what survives that.

A rep can overrule the rules, and the card is replayed, not rebuilt. A `needs_review` card carries all three destinations in one row - Promote to Tier 1, Matched, Not a fit - because a reviewer's job is to decide, not to pick from two of the three answers. When they do, 06 does not assemble a new Slack message. It reloads the decision 01 stored in `oak_deliveries.raw_json`, swaps the classification, drops the routing, and calls 03 with it. 03 then resolves the channel from the policy and attaches the buttons that classification deserves, so a lead promoted to Tier 1 arrives in `#booth-hot` as a full card and a lead ruled out arrives in `#booth-out-of-scope` with no buttons and no mention. One card builder, one channel map, one place to change either.